

**Правительство Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет «Высшая школа экономики»**

Факультет компьютерных наук
Образовательная программа бакалавриата 01.03.02 «Прикладная математика и информатика»

**ОТЧЕТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

в Департаменте программной инженерии Факультета компьютерных наук НИУ ВШЭ

Выполнил студент:
группы БПМИ

(подпись, ФИО)

Проверил:
Руководитель практики от ФКН

Хайбулин Даниэль Ильвович
Преподаватель ФКН НИУ ВШЭ

(оценка, подпись, дата)

Содержание

1	Введение	4
2	Архитектура конкурентных и параллельных вычислений в Python	5
2.1	Многопоточная загрузка сетевых ресурсов	5
2.2	Исследование состояния гонки и синхронизация потоков	5
2.3	Использование многопроцессорного пула для CPU-bound задач	6
3	Асинхронное программирование на базе asyncio	7
3.1	Управление асинхронными задачами через TaskGroup	7
3.2	Обработка множественных исключений	7
3.3	Ограничение времени выполнения и интеграция синхронного кода	7
4	Разработка REST API на FastAPI и валидация данных	8
4.1	Проектирование CRUD интерфейса книжного магазина	8
4.2	Валидация входных схем с помощью Pydantic	8
4.3	Кастомная обработка ошибок	8
5	Оптимизация производительности и тестирование веб-сервисов	9
5.1	Исправление ошибок производительности	9
5.2	Автоматическое тестирование	9
6	Заключение	10

1 Введение

В современной индустрии разработки программного обеспечения создание эффективных, масштабируемых и отказоустойчивых сетевых сервисов является одной из приоритетных задач. Настоящий отчет посвящен прохождению учебной практики на базе мини-курса по сетевому программированию на языке Python.

Основной целью практики является закрепление теоретических знаний и приобретение практических навыков проектирования и реализации конкурентных, асинхронных приложений и REST-сервисов с использованием современных технологий и фреймворков.

Для достижения поставленной цели в процессе практики решались следующие задачи:

1. Изучение и практическое применение инструментов параллельных вычислений в Python (многопоточность, многопроцессорность, синхронизация потоков, устранение состояния гонки).
2. Освоение парадигмы асинхронного программирования (модуль `asyncio`, событийный цикл, асинхронные группы задач, обработка параллельных исключений).
3. Проектирование и разработка клиент-серверной архитектуры по протоколу HTTP с использованием современного веб-фреймворка FastAPI.
4. Проведение валидации сложных структур данных с помощью Pydantic и реализация гибкой системы обработки ошибок.
5. Разработка автоматических тестов для проверки работоспособности, корректности и производительности разработанных сетевых интерфейсов.

Актуальность данной работы обусловлена широким распространением асинхронных технологий в бэкенд-разработке и необходимостью понимания механизмов параллельного исполнения кода для оптимального использования вычислительных ресурсов серверов.

2 Архитектура конкурентных и параллельных вычислений в Python

В рамках первой лекции и соответствующего домашнего задания были подробно изучены механизмы организации конкурентных вычислений в языке Python. Основное внимание уделялось преодолению ограничений GIL (Global Interpreter Lock) и правильному выбору абстракции для ввода-вывода (I/O-bound) и вычислительно сложных (CPU-bound) задач.

2.1 Многопоточная загрузка сетевых ресурсов

Для работы с сетевыми запросами (I/O-bound задачи) наиболее эффективным подходом является использование потоков. В ходе практики была спроектирована система загрузки веб-страниц по списку URL-адресов.

Для управления жизненным циклом потоков применялся класс `ThreadPoolExecutor` из стандартного модуля `concurrent.futures`. Использование пула потоков позволяет снизить накладные расходы на создание и уничтожение системных потоков за счет их переиспользования.

Была реализована логика сбора результатов по мере их готовности (с использованием функции `as_completed`), обработка исключений (сетевые ошибки, таймауты выполнения) и механизм обратного вызова (`callback`) для отслеживания прогресса скачивания в реальном времени. Такой подход позволил значительно сократить суммарное время ожидания ответов от серверов по сравнению с последовательным выполнением запросов.

2.2 Исследование состояния гонки и синхронизация потоков

При работе с общей памятью в многопоточной среде возникает фундаментальная проблема конкурентного доступа — состояние гонки (`race condition`). В рамках работы была смоделирована ситуация, в которой несколько потоков одновременно инкрементировали общий целочисленный счетчик без явной синхронизации, что приводило к потере данных и некорректным результатам.

Для обеспечения потокобезопасности (`thread safety`) был использован примитив синхронизации `threading.Lock`. Был разработан класс `BankAccount`, реализующий операции пополнения, списания и проверки баланса. Все операции, связанные с чтением и записью общего состояния, были вынесены в критическую секцию, защищенную блокировкой:

```
with self.lock:
    # атомарная операция модификации баланса
```

Применение блокировок позволило гарантировать консистентность данных при одновременной работе произвольного количества конкурирующих потоков.

2.3 Использование многопроцессорного пула для CPU-bound задач

Для задач, требующих больших вычислительных мощностей процессора (CPU-bound), стандартные потоки в Python не приносят ускорения из-за работы GIL, который разрешает только одному потоку исполнять байт-код Python в один момент времени.

Для обхода этого ограничения был применен модуль `multiprocessing`. Был разработан модуль параллельных вычислений на базе `multiprocessing.Pool`. За счет создания отдельных процессов, каждый из которых имеет собственный интерпретатор Python и адресное пространство, вычисления выполняются истинно параллельно на разных ядрах процессора. Была организована корректная передача данных между процессами посредством сериализации (`pickle`) и упорядочивание результатов вычислений в соответствии с исходным порядком входных данных.

3 Асинхронное программирование на базе `asyncio`

Модуль `asyncio` предоставляет возможность написания конкурентного кода с использованием синтаксиса `async/await`. Асинхронность является стандартом де-факто для высоконагруженных сетевых приложений, так как позволяет обрабатывать тысячи одновременных соединений в рамках одного системного процесса и потока.

3.1 Управление асинхронными задачами через `TaskGroup`

В ходе практики были изучены современные методы управления жизненным циклом корутин. Вместо устаревшего метода `asyncio.gather` был применен структурированный подход на основе `asyncio.TaskGroup` (введенный в Python 3.11).

`TaskGroup` гарантирует, что если одна из запущенных задач завершается с ошибкой, все остальные активные задачи группы автоматически отменяются, предотвращая утечку ресурсов и зависание фоновых процессов. Были реализованы механизмы безопасного выполнения сетевых запросов с индивидуальным перехватом исключений для каждой задачи внутри группы, что предотвращает аварийный останов всей группы при единичном сбое.

3.2 Обработка множественных исключений

С внедрением `TaskGroup` в Python появился новый синтаксис для обработки группы ошибок — `except*`. В рамках выполнения заданий был разработан блок обработки ошибок, позволяющий перехватывать и логировать различные типы исключений (например, `ValueError` и `TimeoutError`), возникшие параллельно в разных асинхронных задачах, без прерывания выполнения основной программы.

3.3 Ограничение времени выполнения и интеграция синхронного кода

Были разработаны механизмы ограничения времени выполнения задач с помощью `asyncio.timeout` и `asyncio.wait`. В случае превышения лимита времени асинхронная задача корректно отменяется, освобождая сетевые сокеты.

Также был решен вопрос интеграции блокирующего синхронного кода в асинхронный событийный цикл. Для предотвращения блокировки event loop использовался запуск синхронных функций в отдельном пуле потоков через `loop.run_in_executor`.

4 Разработка REST API на FastAPI и валидация данных

FastAPI — это современный, быстрый веб-фреймворк для создания API на основе Python. Он построен на базе стандарта ASGI и библиотеки Pydantic, что обеспечивает высокую производительность и автоматическую генерацию документации.

4.1 Проектирование CRUD интерфейса книжного магазина

Во втором домашнем задании было спроектировано и реализовано REST API для управления каталогом книг и категорий. Были разработаны следующие эндпоинты: **GET /books** — получение списка всех книг с возможностью фильтрации по категории и году издания. **GET /books/{id}** — получение детальной информации о книге по ее идентификатору. **POST /books** — добавление новой книги в каталог с проверкой уникальности ISBN. **PUT /books/{id}** — полное обновление данных о книге. **DELETE /books/{id}** — удаление книги из каталога с возвратом статуса **204 No Content**. **GET /categories** и **POST /categories** — управление справочником категорий.

Данные для простоты хранились в оперативной памяти (in-memory) в виде структурированных словарей и списков с потокобезопасным инкрементом идентификаторов.

4.2 Валидация входных схем с помощью Pydantic

Для защиты приложения от некорректных данных были разработаны схемы Pydantic. Модели `BookCreate` и `CategoryCreate` описывают требования к входящим запросам: **Поля `title` и `author` должны быть непустыми строками длиной до 100 символов.** Год издания (`year`) должен находиться в диапазоне от 0 до 2025. **Стоимость книги (`price`) должна быть строго положительным числом.** Поле `isbn` должно соответствовать международному стандарту. Был написан валидатор `@field_validator("isbn")`, проверяющий, что строка после удаления дефисов состоит строго из 10 или 13 цифр.

Использование встроенного механизма валидации FastAPI позволило автоматически отклонять некорректные запросы со статус-кодом 422 `Unprocessable Entity` и детальным описанием ошибок.

4.3 Кастомная обработка ошибок

Для улучшения читаемости ответов API была спроектирована система кастомных исключений: **`BookNotFoundException` — выбрасывается, если запрашиваемая книга отсутствует в базе данных. Возвращает статус `404 Not Found`.** **`DuplicateIsbnException` — выбрасывается при попытке добавить книгу с уже существующим ISBN. Возвращает статус `409 Conflict`.**

С помощью декоратора `@app.exception_handler` были зарегистрированы глобальные обработчики этих ошибок, которые форматируют ответ сервера в единый стандарт: JSON-объект, содержащий понятное текстовое сообщение `detail` и машиночитаемый код ошибки `code` (например, `"NOT_FOUND"` или `"DUPLICATE_ISBN"`).

5 Оптимизация производительности и тестирование веб-сервисов

5.1 Исправление ошибок производительности

В процессе работы над проектом было проведено профилирование и исправление типичных архитектурных ошибок в веб-приложении:

1. **Устранение блокировок событийного цикла:** в эндпоинте `/slow-sync` блокирующий вызов `time.sleep(0.5)` останавливал работу всего однопоточного сервера. Он был заменен на неблокирующий `await asyncio.sleep(0.5)`, что позволило серверу параллельно обрабатывать другие входящие запросы во время ожидания задержки.
2. **Предотвращение Race Condition при подсчете метрик:** инкремент глобального счетчика обращений к эндпоинтам был защищен блокировкой `threading.Lock`, чтобы предотвратить потерю логов при одновременных обращениях из нескольких потоков.
3. **Обработка критических сбоев в бизнес-логике:** в эндпоинте деления чисел `/divide` был добавлен перехват `ZeroDivisionError`. Вместо падения сервера с кодом `500 Internal Server Error` приложение теперь возвращает корректный статус `400 Bad Request` с описанием ошибки деления на ноль.

5.2 Автоматическое тестирование

Качество и стабильность разработанного кода были подтверждены написанием автоматических тестов с использованием фреймворка `pytest`.

С помощью тестового клиента `TestClient` (`Starlette`) были покрыты все разработанные эндпоинты. Написанные тесты проверяют: **Возврат правильных статус-кодов при успешных операциях (200, 201, 204)**. Корректность валидации данных (отклонение неверных ISBN, отрицательных цен и годов). **Обработку пограничных случаев (деление на ноль, поиск несуществующих книг, повторное удаление)**. Асинхронное поведение эндпоинта `/slow-sync` (проверка, что два параллельных запроса выполняются конкурентно и не блокируют друг друга).

Всего в рамках практики было успешно пройдено **47 комплексных тестов**, подтверждающих полное соответствие разработанного веб-сервиса техническому заданию.

6 Заключение

В ходе прохождения учебной практики были полностью выполнены все поставленные задачи. Были глубоко изучены и отработаны на практике механизмы конкурентного программирования на Python (потoki, процессы, асинхронность), а также проектирование современных REST API сервисов с помощью FastAPI и Pydantic.

Полученные навыки проектирования баз данных в памяти, валидации входящих запросов, обработки исключений и автоматического тестирования приложений закладывают прочный фундамент для дальнейшей разработки сложных распределенных информационных систем и высоконагруженных веб-сервисов. Все исходные коды программ и тестов были успешно зафиксированы в системе контроля версий Git и отправлены в удаленный репозиторий.